

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра “Вычислительная техника”

Отчёт

по лабораторной работе №3
по курсу “Программирование на языке Java”
на тему “Обработка исключительных ситуаций”
Вариант 9

Выполнили студенты гр. 23ВВП1:

Лобачева К.С.

Зонь М.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Карамышева Н.С.

Пенза 2026

Цель работы

Изучить механизм обработки исключительных ситуаций.

Лабораторное задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса `Exception`, и генерировать исключение, если возникает попытка создать экземпляр класса `RecIntegral` со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000.

Ход выполнения работы

В программу добавлен класс `ValidData`, отвечающий за исключение при попытке ввести некорректные данные (рис. 1).

```
public class ValidData extends Exception {  
    public ValidData(String message) {  
        super(message);  
    }  
}
```

Рисунок 1 — Реализация класса `ValidData`

В классе `RecIntegral` реализована проверка вводимых данных непосредственно в конструкторах. При создании экземпляра класса выполняется проверка на принадлежность значений диапазону от 0,000001 до 1000000, а также проверка условия, что нижний предел меньше верхнего. При нарушении любого из условий генерируется исключение `ValidData` (рис. 2):

```
public RecIntegral(double lowerLimit, double upperLimit, double step) throws ValidData {  
    if (lowerLimit < 0.000001 || lowerLimit > 1000000) {  
        throw new ValidData("Нижний предел вне диапазона [0.000001, 1000000]");  
    }  
    if (upperLimit < 0.000001 || upperLimit > 1000000) {  
        throw new ValidData("Верхний предел вне диапазона [0.000001, 1000000]");  
    }  
    if (step < 0.000001 || step > 1000000) {  
        throw new ValidData("Шаг вне диапазона [0.000001, 1000000]");  
    }  
    if (lowerLimit >= upperLimit) {  
        throw new ValidData("Нижний предел должен быть меньше верхнего!");  
    }  
    this.lowerLimit = lowerLimit;  
    this.upperLimit = upperLimit;  
    this.step = step;  
    this.hasResult = false;  
    this.result = 0;  
}
```

Рисунок 2 — Проверка на принадлежность диапазону

Для обработки строковых значений, вводимых пользователем в текстовые поля, реализован дополнительный конструктор, принимающий три строки, а также приватный статический метод `parseDouble()`, который преобразует строку в число и генерирует исключение `ValidData` при неудачном парсинге (рис. 3):

```
private static double parseDouble(String value, String fieldName) throws ValidData {
    if (value == null || value.trim().isEmpty()) {
        throw new ValidData("Поле '" + fieldName + "' не заполнено!");
    }
    try {
        return Double.parseDouble(value.trim().replace(',', '.'));
    } catch (NumberFormatException e) {
        throw new ValidData("Поле '" + fieldName + "' " + value + "' не является числом!");
    }
}
```

Рисунок 3 — Метод `parseDouble`

Обработка исключений производится в блоке `try-catch` метода `Vnesti()` главного класса `l2frame`. При нажатии кнопки "Внести" создается объект `RecIntegral`, и если данные некорректны, исключение перехватывается и выводится диалоговое окно с сообщением об ошибке (рис. 4):

```
try {
    RecIntegral rec = new RecIntegral(
        jTextField2.getText(), // нижний предел
        jTextField1.getText(), // верхний предел
        jTextField3.getText()  // шаг
    );

    tableModel.addRow(new Object[]{
        rec.getLowerLimit(),
        rec.getUpperLimit(),
        rec.getStep(),
        ""
    });

    dataList.add(rec);

    jTextField1.setText("");
    jTextField2.setText("");
    jTextField3.setText("");
} catch (ValidData e) {
    javax.swing.JOptionPane.showMessageDialog(this,
        e.getMessage(),
        "Ошибка ввода",
        javax.swing.JOptionPane.ERROR_MESSAGE);
}
```

Рисунок 4 — Обработка исключений

Результат работы программы представлен на иллюстрациях 5-8.

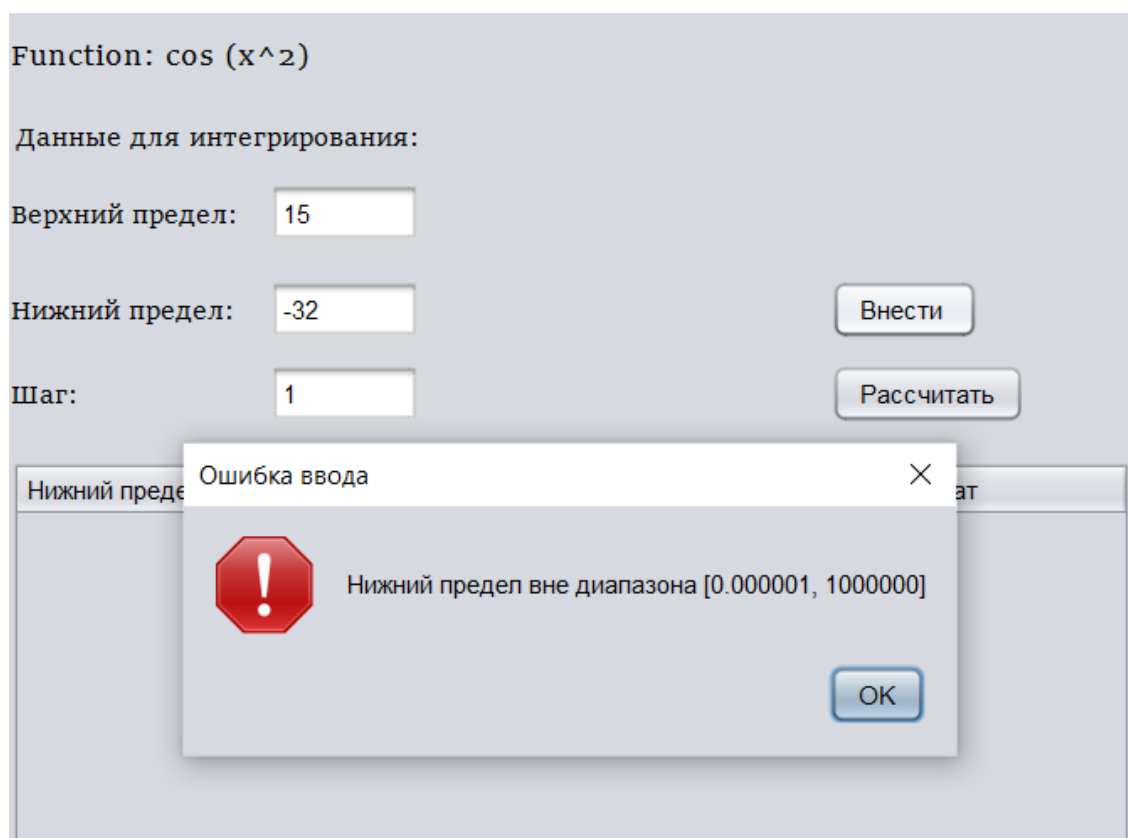


Рисунок 5 — Проверка нижнего предела на принадлежность диапазону

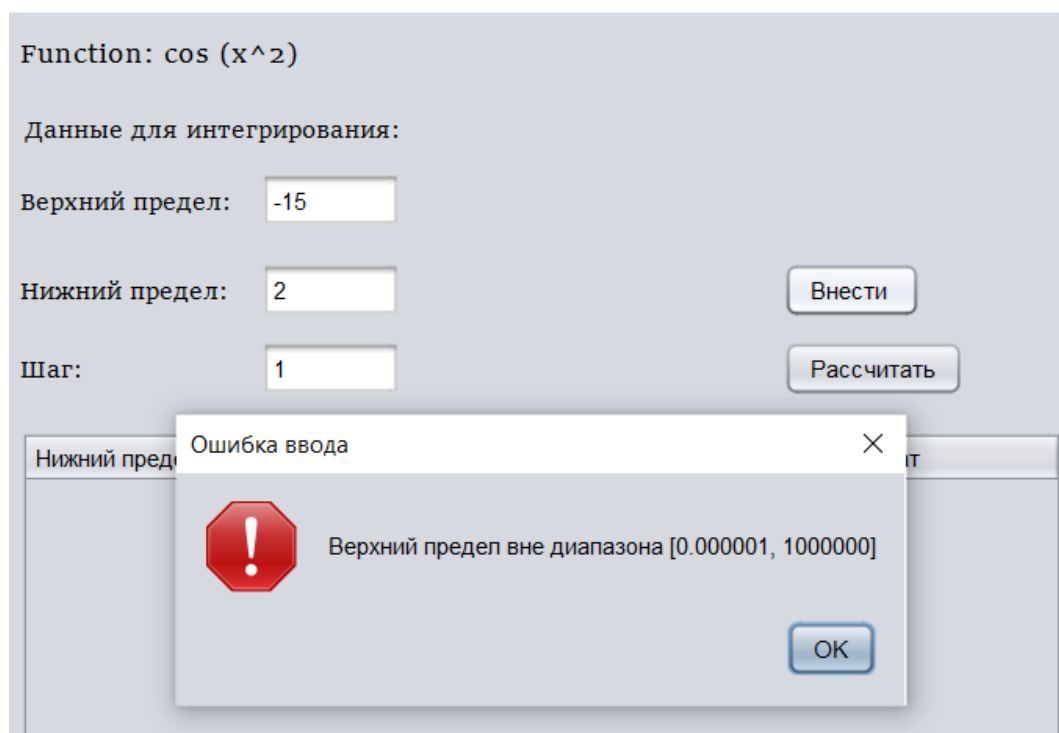


Рисунок 6 — Проверка верхнего предела на принадлежность диапазону

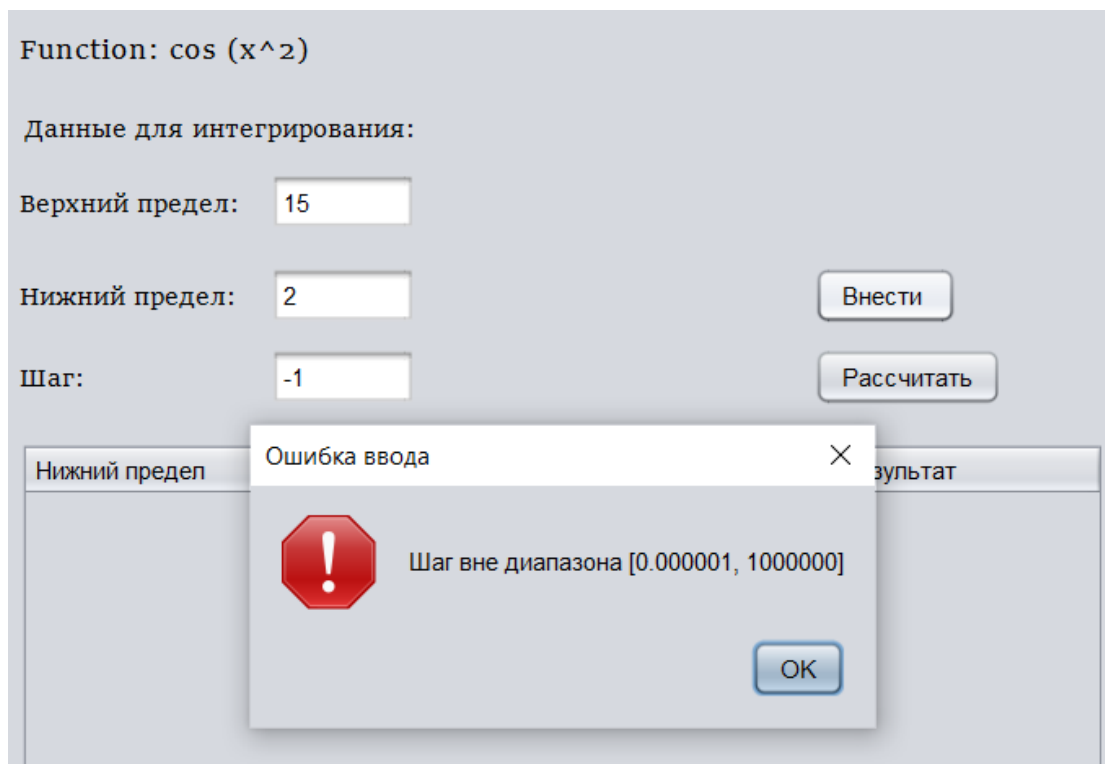


Рисунок 7 — Проверка шага на принадлежность диапазону

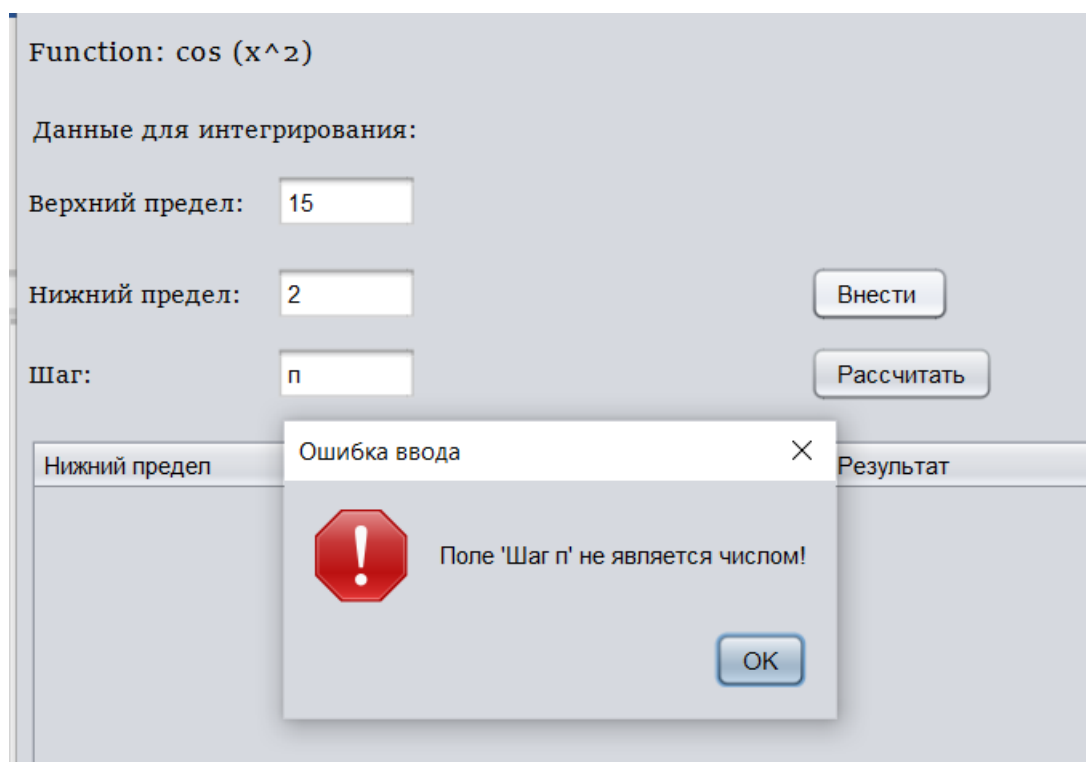


Рисунок 8 — Проверка на числовое значение

Вывод

В ходе выполнения данной лабораторной работы были получены навыки организации собственных исключений и их обработки на языке Java.